

COMPUTER SCIENCE TRIPOS Part IB – 2024 – Paper 4

7 Cybersecurity (fms27)

Some naïve teenagers want to attack a web application that does not salt its users' passwords, but simply hashes them. Hoping they will someday obtain the file of digests on the dark web, they want to precompute a compressed lookup table that will let them quickly crack all lowercase alphanumeric passwords of up to 15 characters once they get the file. They implement precomputed hash chains, but they ignore the possibility of collisions because they do not understand the issue. Their table must fit in their available disk space s . Each digest is 16 bytes long.

Please use the following symbol names and use sensible approximations where needed.

l_a	length of alphabet (charset size) for targeted passwords	$26 + 10 = 36$
l_d	length of one digest, in bytes	16
l_p	length of password, in bytes (max len to be explored)	15
s	storage space available for the compressed table, in bytes	8×10^{12}
t_h	time to compute a hash on attackers' CPU, in seconds	10^{-5}

(a) Under the attackers' incorrect assumption that hash collisions may be ignored:

- (i) Derive formulae for the maximum number n_c of chains in the table, and for the minimum length l_c (in passwords) of each chain. Also calculate numerical values for these quantities. (*4 results.*) [4 marks]

Answer: Each chain takes up the space of its key (final digest) and value (starting password). Available disk space divided by space of one chain gives maximum number of chains.

$$n_c \cdot (l_d + l_p) \leq s$$

$$\boxed{n_c \leq \frac{s}{l_d + l_p}}$$

$$n_c \leq 8e12 / (16 + 15) = \boxed{2.6e11}$$

Each chain contributes l_c passwords (distinct under the attackers' mistaken assumption that collisions may be ignored). The total number of passwords covered by the table is the total number of passwords in all chains, and this must equal or exceed the size of the sought password space.

$$l_c \cdot n_c \geq l_a^{l_p}$$

$$\boxed{l_c \geq l_a^{l_p} / n_c} = 36^{15} / 2.6e11 = 2.2 \text{ e}23 / 2.6e11 = \boxed{8.5e11}$$

- (ii) Give a clear and full description of the algorithm for recovering the password that is the preimage of a given digest d . Pseudocode is not required for full marks, but will be rewarded if it adds clarity and precision. [5 marks]

Answer: Each entry in the table represents one chain. The table maps keys to values: given the final digest of a chain (the key), the table lets you look up the initial password (the value) with which you may re-generate the whole chain. *Hashing* a password

transforms it into a digest; *reducing* a digest transforms it into a password. Repeating this pair of steps l_c times (l_c times “minus half” to be pedantic—let’s not be) transforms the initial password into the final digest.

The process of retrieving the preimage of a given digest consists of two phases. First, identify which chain contains the given digest, by repeatedly reducing it and hashing it and, at every step, checking whether the digest appears among the keys of the table. If the digest appears in any chain, in at most l_c steps it must be transformed into the digest at the end of the chain (one of the keys of the table). Therefore, if we repeat enough times, either we conclude that the digest is not in the table (since the “precomputed hash chains” method cannot in fact guarantee to cover the whole password space) or we find the chain that contains it.

In the second phase we reconstruct the relevant chain from its beginning until we reach our digest. When we do, we look at the preimage that generated it, which is our sought result.

The above would already get the full marks, but below is some optional pseudocode for completeness.

```
def preimageOf(d)
    "Take digest d and return password p such that hash(p) == d"

    # 1. Find which chain contains d (if any)
    digest = d
    iterations = 0
    found = false
    while iterations < lc:
        if digest in table.keys:
            found = true
            break
        digest = hash(reduce(digest))
        iterations = iterations + 1
    if not found:
        raise DigestNotFoundInTableException(d)

    assert: d is in the chain that starts at password table[d]

    # 2. Find the predecessor of d in that chain
    password = table[d]
    while true:
        digest = hash(password)
        if digest == d:
            return password
        password = reduce(digest)
```

- (iii) Derive a formula for the minimum number n_h of hashes to be computed to build the compressed table, and calculate its numeric value. (*2 results.*) [2 marks]

Answer: To be able to recover each possible password in the sought space, the attackers must hash each of those passwords at least once.

$$\boxed{n_h > l_a^{l_p}} = 36^{15} = \boxed{2.2\text{e}23}$$

- (iv) Use the n_h value from Part (a)(iii) to calculate the time t_s needed to compute the whole table sequentially, assuming one hash computation takes time t_h on the attackers' CPU. Then imagine parallelising the work in three ways: moving from CPU to GPU, using a bank of GPUs, or using a distributed pool of collaborators. Give a reasonable speedup factor for each. Compute how long it would take to build the compressed table if all three speedups were adopted. (5 results.) [5 marks]

Answer: With chains, for every hash I also need to do a reduction. Let's assume the reduction takes comparable time to a hash.

$$t_s = n_h \times t_h \times 2 = 2.2e23 \times 1e-5 \times 2 = 4.4e18 \text{ seconds} = 5e13 \text{ days} = \boxed{1.4e11 \text{ years}}$$

Note in passing (not required for marks) that this is about ten times greater than the age of the universe, estimated at about $1.4e10$ years.

3 ways to parallelize:

- (A) Use a GPU, which is inherently parallel, rather than a CPU. Expected speed-up: around $\boxed{1000 - 10000}$.
- (B) Use a bank of GPUs rather than one. Expected speed-up: around $\boxed{3 - 10}$.
- (C) Use a distributed pool of collaborators (each with own GPU bank... yeah, right). Expected speed-up: around $\boxed{10 - 100}$.

Total maximum speed-up = product of the three maximum estimates = $1e4 \times 1e1 \times 1e2 = 1e7$, and we've been optimistic. It would still take at least $1.4e4 = \boxed{14,000 \text{ years}}$, which is still a non-starter (and we are not even talking about the electricity bill of each of these poor collaborators).

- (b) Briefly explain what hash collisions are and why they matter. How would the presence of collisions invalidate the answer to Part (a)(iii)? [4 marks]

Answer: A collision occurs when two distinct passwords p_1 and p_2 hash to the same digest.

Merged chains: When p_1 and p_2 appear in different chains, after the collision the chains merge, in the sense that they both produce the same sequence of passwords and digests.

Cycles: When p_1 and p_2 appear in the same chain, the chain loops back on itself, in the sense that the sequence of passwords and digests produced after the collision is a repeat of what had already been produced between the occurrence of the first and second of the two colliding passwords.

The answer to Part (a)(iii) assumes that a chain of length l_c will contribute l_c distinct (password, digest) pairs.

With collisions, both merged chains and cycles cause repetitions that reduce the number of distinct (password, digest) pairs contributed by the chain(s) involved in the collision. This means that more chains would be needed to cover the same password space.

In other words, the table generated under the incorrect assumption that collisions may be ignored will be able to recover only a small proportion of the passwords in the password space that the attackers were hoping to cover. The algorithm from Part (a)(ii) will raise many more "not found" exceptions than expected.
